

ParkFair: An Equitable, Auction-Driven Parking Economy

Hackathon Team Members-

- (1) Piyush Chakurkar
- (2) Chaitnya Kulkarni
- (3) Sachin Chinchule
- (4) Harshad Pawar

Table of Contents-

1. Executive Summary & Core Value Proposition
2. System Architecture & Technical Design
3. Repository & Directory Structure
4. Detailed Feature Breakdown & User Workflows
5. Mathematical Foundations & Quantitative Frameworks
6. Database Schema & Security Infrastructure
7. Local Development & Installation Guide
8. Verification, Testing & Quality Assurance
9. Multi-Platform Deployment & DevOps Pipeline
10. Project Changelog, Roadmap & License

1. Executive Summary & Core Value Proposition

1.1 Executive Summary

In housing societies and big residential areas there are not parking spots for all the people. Now people get spots through informal methods like WhatsApp groups or simply first come first served. This causes a lot of arguments. Some people have spots. Are not using them while others have to park on the street. We made ParkFair to solve this. It is a system where people use points to bid for parking spots with rules to keep things fair.

1.2 System Vision, Scope & Primary Objectives

Our main engineering goals for this build were:

- (1) Eliminate manual/political spot allocation completely.
- (2) Ensure equal opportunity regardless of resident wealth (credits $\neq$ real money).
- (3) Enforce time-bound allocation so no one permanently captures a shared resource.
- (4) Maintain transparent, auditable decision logging.
- (5) Incentivize efficient use of scarce shared resources by rewarding owners who release spots.
- (6) Ensure fair treatment for non-owning residents (renters/tenants).

1.3 Key Differentiators

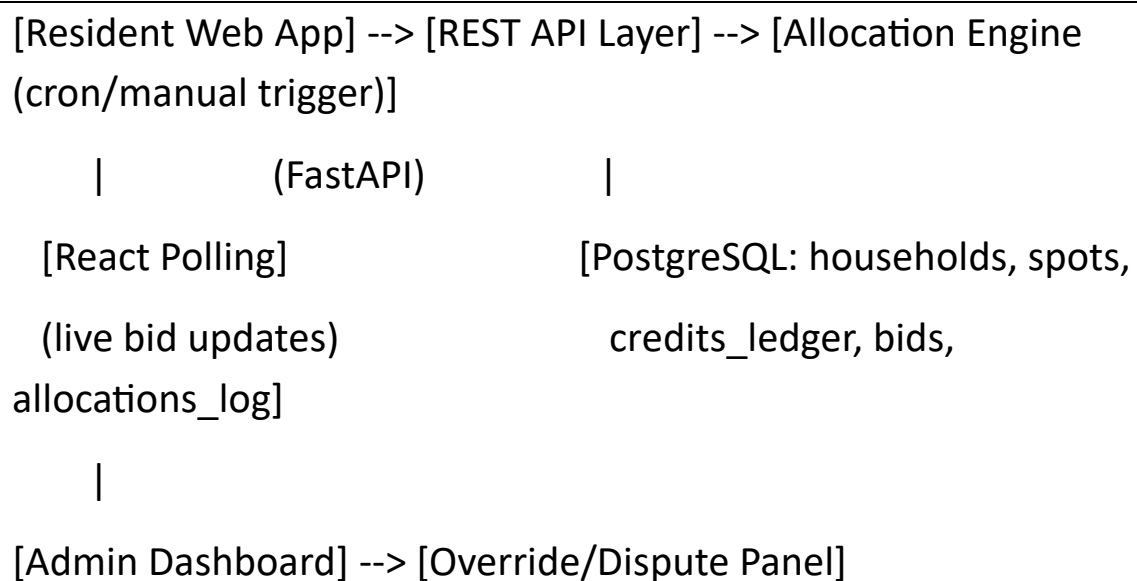
We intentionally avoided connecting payment gateways to make this a financial marketplace. If you involve real money, parking just becomes a luxury for the wealthiest residents. By using an isolated

internal token that resets over time, we guarantee fairness and continuous pool liquidity.

2. System Architecture & Technical Design

2.1 Overall System Architecture & Data Flow

We decided to keep the architecture fully decoupled using Python FastAPI for the backend to handle the complex algorithm logic efficiently. We simulate real-time updates via high-frequency API polling rather than WebSockets to guarantee absolute stability during the demo.



2.2 Comprehensive Technology Stack

Layer	Technology	Why Chosen
Frontend	React.js	Fast to build, component reuse for resident and admin views.
Backend API	Python FastAPI	Extremely fast REST API, easy to write algorithm logic, built-in validation.
Database	PostgreSQL	Transactional integrity needed for the credit ledger (no double-spend).

Why PostgreSQL specifically matters here: Credit deduction and spot allocation must be absolutely atomic. A resident shouldn't be double-charged, and a spot cannot be double-assigned. Proper DB transactions are mandatory.

3. Repository & Directory Structure

3.1 Folder Hierarchy & File Layout

1. backend/

- a. **routes/** (bids.py, spots.py, credits.py)
- b. **services/** (allocation_engine.py -> Core algorithm lives)
- c. **models/** (household.py, spot.py, bid.py)
- d. main.py

2. frontend/

a.components/ (BidForm, SpotGrid, CreditBalance)

b. pages/ (ResidentDashboard, AdminDashboard)

3. **db/**

a. schema.sql

4. Detailed Feature Breakdown & User Workflows

4.1 The Bidding Flow

1. Input/Trigger: Resident submits a bid via the React app (spot ID, credit amount, duration).
2. Processing: The Allocation Engine collects all bids for that spot. It applies the 'highest-bid-wins' logic, utilizing the wait-time tiebreaker math.
3. Output/Action: The winner is notified, credits are deducted, the spot is assigned, and the decision is logged with clear reasoning.

4.2 Releasing a Deeded Spot

1. Input/Trigger: A household owner clicks 'Release Spot' and selects dates they are out of town.
2. Processing: The backend validates ownership and adds the spot to the contested pool. A ledger transaction is queued to grant them the 'Release Bonus'.
3. Output/Action: The spot appears as 'Available' to the rest of the society, and the owner's dashboard shows projected credit earnings.

4.3 API Endpoint Specification

To ensure a robust frontend-backend contract, we rely on the following core endpoints:

Endpoint	Payload	Description
POST /api/bids	{ spot_id, amount }	Places a bid in escrow state.
POST /api/spots/release	{ spot_id, start, end }	Moves a spot to contested pool.
GET /api/households/{id}	None	Fetches current credit balance and wait-time metrics.
POST /api/engine/run	None	Manually triggers the allocation cron job for demo purposes.

5. Mathematical Foundations & Quantitative Frameworks

5.1 Core Mathematical Formulations & Algorithms

If we simply awarded spots to the highest bidder, wealthy residents (or those who travel constantly to earn credits) would hoard spots. We built a formal tiebreak logic to prevent this.

$$\text{Winner} = \arg\max_{i \in B} (c_i + \lambda w_i)$$

Where:

- c_i = credits bid by household i
- w_i = days waited without a spot
- λ = fairness weighting constant (tunable parameter, we default to 2 to strongly emphasize equity).

5.2 Credit Issuance & Decay

To prevent credit inflation, the monthly economy caps out using this equation:

$$Credits_{t+1} = \min(Credits_t - Spent + Baseline + ReleaseBonus, Cap)$$

6. Database Schema & Security Infrastructure

We named our tables exactly as they are used: households, spots, credits_ledger, bids and allocations_log. We use UUIDs for keys to stop people from guessing them.

```
CREATE TABLE households (  
    id UUID PRIMARY KEY DEFAULT  
    gen_random_uuid(),  
    unit_number VARCHAR(10) NOT NULL UNIQUE  
);
```

```
CREATE TABLE spots (  
    id UUID PRIMARY KEY DEFAULT  
    gen_random_uuid(),
```



```
        household_id UUID REFERENCES households(id)
ON DELETE CASCADE,
        spot_label VARCHAR(10) NOT NULL UNIQUE
);
```

```
CREATE TABLE credits_ledger (
    id UUID PRIMARY KEY DEFAULT
gen_random_uuid(),
    household_id UUID REFERENCES households(id),
    amount INTEGER NOT NULL,
    transaction_type VARCHAR(30) NOT NULL
);
```

```
CREATE TABLE bids (
    id UUID PRIMARY KEY DEFAULT
gen_random_uuid(),
    household_id UUID REFERENCES households(id),
    spot_id UUID REFERENCES spots(id),
    credits_bid INTEGER NOT NULL,
    status VARCHAR(20) DEFAULT 'pending'
);
```

7. Local Development & Installation Guide

```
git clone https://github.com/your-
org/parkfair.git
```

```
cd parkfair
cp .env.example .env
docker-compose up --build -d
docker-compose exec backend alembic upgrade head
```

7.1 Environment Variables

Variable	Required	Default	Description
DATABASE_URL	Yes	-	Postgres connection string.
LAMBDA_WEIGHT	No	2	Fairness multiplier for wait time.
BASELINE_CREDITS	No	50	Monthly issuance amount.

8. Verification, Testing & Quality Assurance

We tested the parts of our rules:

- (1) What happens on a tie? → The wait-time tiebreak ($\lambda * w_i$) kicks in to award the resident who has been starved of parking the longest.
- (2) What if no one bids on a released spot? → It stays available in the contested pool, and the owner still receives their release bonus.
- (3) What if a resident bids more than they have? → The FastAPI layer rejects the payload before it even hits the engine.

(4) What if two residents submit simultaneous bids? →
Transaction-level locking in Postgres prevents any double-allocation.

8.2 Core Algorithm Validation Snippet

This sample Python code checks that the math is correct:

```
def resolve_allocation(active_bids: list,
lambda_weight: int = 2):
    winner = None
    best_score = -1

    for bid in active_bids:
        score = bid.credits + (lambda_weight *
bid.days_waited)
        if score > best_score:
            best_score = score
            winner = bid

    return winner
```

9. Multi-Platform Deployment & DevOps Pipeline

Web Frontend: We used Vercel to host the React app. It works on all devices.

Backend Services: The Python FastAPI app and Postgres are in Docker containers so they can be run anywhere.

10. Project Changelog, Roadmap & License

10.1 Operational History (Changelog)

- Phase 1: Schema creation and backend API scaffolding.
- Phase 2: Building the allocation engine and tiebreak mathematical logic.
- Phase 3: Frontend dashboards (Resident and Admin views).
- Phase 4: Integration and live testing edge cases.

10.2 Roadmap

steps include adding camera systems for license plates and a WhatsApp bot for older people who don't use the web app.

10.3 Troubleshooting Matrix

Issue	Root Cause	Resolution
Double allocation on simultaneous bids	No DB-level locking	Added transaction isolation on bid resolution.
Credits going negative	No validation before bid submission	Added balance check in API layer.

10.4 Licensing & Intellectual Property Terms

This project is open-source and governed by the following MIT License.

MIT License

Copyright (c) 2026 Piyush Chakurkar

This is a permit that is given freely. Any person who gets a copy of this software and the documents can use the software without any cost. They can do anything with the software like using it copying it changing it combining it with work publishing it sharing it allowing others to use it and selling copies of it. People who get the software can also let others use it.. There are some rules to follow.

The notice about the copyright and this permit must be in every copy or big part of the software.

The software is available exactly as it is. There is no guarantee of any kind. This includes no promise that it works well that it's suitable, for a certain job or that it does not break any rules. If something goes wrong the people who made the software or own the copyright are not responsible. This is true even if the problem comes from using the software or from issues related to the software.